

Mid Term report

Team 18

November 2025

Contents

1	Introduction	2
2	Progress Update	2
3	Experimentation	2
3.1	Feature Engineering and Selection	2
3.1.1	Resampling	2
3.1.2	Normalization	2
3.1.3	Feature Selection Process	2
3.2	Reinforcement Learning and Expert-Guided Setup	3
3.3	Transition to Stochastic Dynamic Programming (SDP)	3
3.4	Fractal Brownian Bridge Construction	4
3.5	Behavioral and Quantitative Observations	4
3.6	Summary	4
4	Strategy Overview	5
4.1	Data Processing and Feature Engineering	5
4.2	PPO Agent Architecture	5
4.3	Reward Engineering: The “Disciplined Trader” Reward	5
4.4	Resulting Behaviour of the Agent	5
5	Performance Metrics	6
5.1	EBY Data Performance	6
5.2	EBX Data Performance	6
6	Future Scope	6
6.1	Reward and Behavior Engineering	6
6.2	Risk Management and Stop Loss	7
6.3	Better Feature Exploration	7

1 Introduction

This document is the midterm evaluation report presented by **Team 18**. This report includes the progress update, experimentation, strategy overview, performance metrics, and the future scope before end evaluation.

2 Progress Update

Our first approach was using a PPO reinforcement learning agent on a large set of market features, but the agent consistently learned not to trade. This indicated that the raw features did not provide a strong enough predictive edge to justify taking positions in a noisy intraday environment.

To improve this, an expert-guided approach using a Stochastic Dynamic Programming (SDP) solver was introduced to provide optimal actions over a predicted price path. However, the agent again remained inactive. The short-term penalties experienced during trades outweighed the long-term gains, making the expert trajectory appear unfavourable from the agent's perspective. We then tested the SDP solver as a standalone strategy. Under ideal information it performed extremely well, confirming the strength of its execution logic. But when paired with our causal Fractal Brownian Bridge predictions, performance deteriorated because the predictions lagged at market turning points. This caused the solver to hold incorrect positions during reversals, leading to large losses.

With prediction lag identified as a central issue, we focused on feature engineering to improve the underlying signal. By selecting a smaller and more informative feature set, the PPO agent finally became active and began taking high-conviction trades aligned with genuine market structure. Final backtests showed that the agent captured strong opportunities and generated meaningful alpha, but also suffered sharp drawdowns. The agent held losing trades with the same conviction as winning ones, allowing occasional losses to erase accumulated gains.

Overall, the system has progressed from complete inactivity to a high-conviction, alpha-generating strategy. The remaining task is to implement explicit risk controls so the agent can preserve profits and exit losing positions more effectively.

3 Experimentation

This is the third section. Here, you can present the results of your work. You may want to include figures, tables, or charts to display your findings. Following the presentation of results, you can discuss their implications and what they mean in the context of your project.

3.1 Feature Engineering and Selection

To improve the signal quality and provide the agent with a more decisive view of the market, a detailed feature engineering process was undertaken.

3.1.1 Resampling

Tick-by-tick financial data is highly volatile and computationally intensive. To create a smoother and more meaningful representation, we aggregated the data into **per-minute intervals**. The resampling methods were tailored to each feature category:

- **Price-based Features:** Resampled into OHLC (Open, High, Low, Close) format for each minute.
- **Price Behavior (PB) Features:** The **mean** value within each one-minute interval was used.
- **Band Behavior (BB) Features:** The **last** available value in the interval was retained.
- **Volume-based Features:** The **sum** of all tick-level volumes within the minute was taken to represent total activity.
- **Other Derived Features:** Resampled using the **mean** value.

3.1.2 Normalization

To ensure all features were on a consistent scale, a **Standard-Scaler** was trained on the first 10-20 days of data. The resulting scaling parameters (mean and standard deviation) were then applied to the entire dataset, making the features comparable across different time periods and enhancing model stability.

3.1.3 Feature Selection Process

Feature selection was performed in multiple stages to retain only the most predictive and informative features.

Time-Lagged and Cross-Feature Analysis Very short time-lagged features (below T4) were discarded as they primarily captured noise. We then generated derived features using time-lagged differences within the same feature family (e.g., $PB1_{T9} - PB1_{T7}$) to model the evolution between fast and slow-moving signals.

The discriminative power of each feature was evaluated by observing its ability to separate optimal long and short trading signals. Features that showed clear thresholds for separating bullish and bearish clusters were retained.

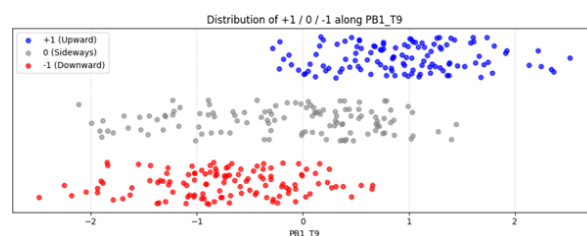


Figure 1: Feature-based separation of long and short signals.

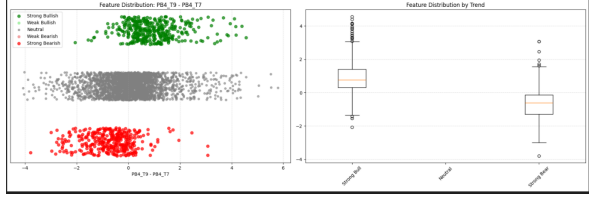


Figure 2: Example of a feature with strong discriminative power.

Volatility and Volume-Based Evaluation For volatility and volume features, we calculated feature-wise accuracy scores based on how well they identified profitable trades within specific market regimes. The top-performing features from all categories were selected for the final feature set.

Intra-Feature Transformations To capture momentum and reversal dynamics, we also generated intra-feature time-lagged differences (e.g., $PB_{i_T7} - PB_{i_T7.shift(-k)}$). This helped identify distinct regions of trend-following and mean-reversion behavior.

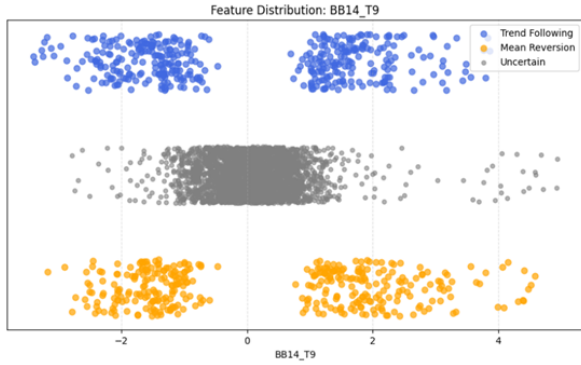


Figure 3: Identifying trend-following and mean-reversion regions.

By analyzing feature clusters, we selected the top N features that most clearly differentiated between these market behaviors.

TOP 10 INDIVIDUAL FEATURES:				
feature	family	long_accuracy	short_accuracy	combined_accuracy
BB14_T6	BB14	0.887468	0.876404	0.881936
BB14_T4	BB14	0.884910	0.873596	0.879253
BB5_T6	BB5	0.882353	0.870787	0.876570
BB13_T6	BB13	0.882353	0.870787	0.876570
BB14_T5	BB14	0.882353	0.870787	0.876570
BB5_T5	BB5	0.874680	0.862360	0.868520
BB13_T5	BB13	0.874680	0.862360	0.868520
BB15_T6	BB15	0.872123	0.859551	0.865837
PB9_T4	PB9	0.869565	0.856742	0.863153
PB9_T5	PB9	0.869565	0.856742	0.863153

TOP 10 DIFFERENCE FEATURES:				
feature	family	long_accuracy	short_accuracy	combined_accuracy
PB9_T9-T8	PB9	0.869565	0.856742	0.863153
PB9_T7-T5	PB9	0.869565	0.856742	0.863153
BB14_T7-T5	BB14	0.797954	0.778090	0.788022
BB5_T7-T5	BB5	0.787724	0.766854	0.777289
BB13_T7-T5	BB13	0.787724	0.766854	0.777289
BB15_T7-T5	BB15	0.782609	0.761236	0.771922
BB14_T9-T8	BB14	0.774936	0.752809	0.763873
BB5_T9-T8	BB5	0.769821	0.747191	0.758506
BB13_T9-T8	BB13	0.769821	0.747191	0.758506
BB15_T9-T8	BB15	0.769821	0.747191	0.758506

Figure 4: Selection of Top-K features on basis of accuracy DIFFERENCE FEATURES and INDIVIDUAL FEATURES.

3.2 Reinforcement Learning and Expert-Guided Setup

We implemented a Proximal Policy Optimization (PPO) RL agent for full-position trading, but the model quickly converged to inactivity due to insufficiently decisive features. High volatility discouraged exploration, while excessive smoothing removed useful price movement, highlighting the need to filter noise without losing tradable structure. To improve guidance, we introduced an expert-assisted PPO framework using a Dynamic Programming module that generated ideal position sequences over predicted future paths. Analysis showed that volatility typically remained within a narrow band, indicating limited short-term predictability outside regime shifts. The agent accordingly learned to trust expert signals in stable periods and act cautiously during volatile transitions. Although Brownian Bridge-based simulations provided a realistic testing environment and performed well in theory, the approach struggled on live data because step-by-step penalties discouraged the agent from following otherwise profitable expert trajectories.

3.3 Transition to Stochastic Dynamic Programming (SDP)

To maximize profits on predicted prices, we used a Stochastic Dynamic Programming (SDP) approach instead of traditional DP. This includes rolling volatility as a doubt factor in place of deterministic DP to handle volatility more naturally. This enhanced risk management and stability during unexpected market conditions.

SDP technique is theoretically strong in identifying high confidence moments and adapting it's behaviour to volatility in the data. During experimentation on ideal data it worked totally fine but it failed while executed on prices predicted using Fractal Brownian Bridge. Performance declined till -45 percent real return because FBB predicted prices possessed a lag (with some confidence). Due to this even with 0.96

correlation between prediction and actual, it led to signals like buying at peak and selling at troughs leading to obvious losses. As a result, our focus switched from model creation to risk management and signal quality.

Subsequent tests with longer planning windows and tweaked transition probabilities gave us only small gains in stability. In the end, they made it clear that the predictive signal itself had a very short-lived edge. That insight confirmed that a trend-following strategy simply didn't fit the behavior of the data.

- Profit Hurdle: Artificially increasing transaction cost filtered out low-conviction trades.
- Skepticism Knob: Amplifying perceived volatility created an adaptive stop-loss, forcing quick exits when trades moved unfavorably.

With these improvements in place, the strategy became noticeably more capital-efficient. It still ended the period with a loss of about -10 percent, but the benchmark was down more than 5 percent, so the performance gap narrowed. During this entire phase, the strategy kept its drawdowns consistently low—between 1 and 3 percent—which pointed to improved stability and tighter risk management.

3.4 Fractal Brownian Bridge Construction

The Fractal Brownian Bridge (FBB) model generated short-horizon price paths that served as the predictive input for the SDP executor. For each interval, a Random Forest Regressor first estimated the endpoint price using the previous three observations, after which stochastic bridges were constructed between the current and predicted values by recursively inserting midpoint estimates with controlled Gaussian noise. This produced realistic trajectories that preserved structural coherence.

From 10,000 such simulations, our criteria of choosing the best bridge was how similar it is to the tail of the last prices since we can't compare with the actual prices to maintain causality. To compare with the tail we use same length head of the constructed bridge. We compare using mathematical quantities. We use the percentage change in DC Offset, log changes in 1st and 2nd order fourier harmonics of the tail and head, since over a small range tail concatenated with head should act like one signal with some particular fourier coefficients. We also use KL divergence to compare between the probability distribution of the tail and head. Cosine similarity is also used to compare the trend following between the two sequences. We use the weighted sum of all these quantities to choose the best bridge.

The chosen bridges achieved a mean absolute error of roughly 0.06, and the causal predictions showed a strong 96 percent correlation with their non-causal counterparts. Despite this accuracy, a small but consistent prediction lag led to substantial execution errors, resulting in an initial loss of about -45 percent when coupled with the SDP. This demonstrated that high predictive accuracy alone is insufficient without proper alignment in timing.

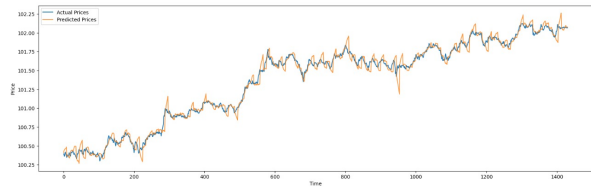


Figure 5: Bullish Market

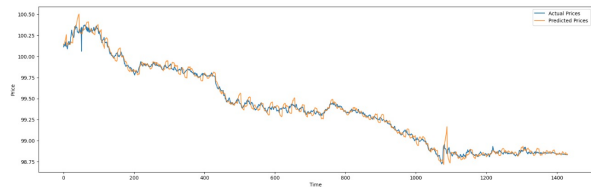


Figure 6: Bearish Market

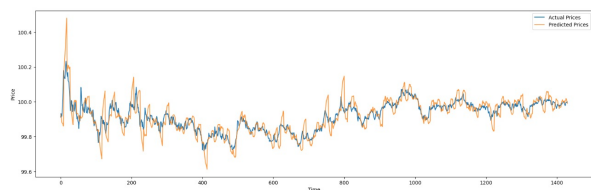


Figure 7: Sideways Market

3.5 Behavioral and Quantitative Observations

The numerical progression across experiments showed clear improvements in behaviour and stability. Net returns rose from -12.5 percent in the initial version to roughly -1 to -3 percent in later iterations, marking a substantial reduction in cumulative loss. From the second iteration onward, all versions outperformed their benchmarks, indicating that the strategy's directional logic was consistently correct even if absolute returns were still slightly negative. Drawdowns narrowed to the 2.5-3.1 percent range, and trading frequency dropped from 8-10 trades per day to about 1.6-1.7, reflecting greater selectivity. The win rate increased to about 44 percent with a favourable payoff skew, showing an improved ability to capture high-reward opportunities. The main outstanding issue is the agent's tendency to hold losing trades too long. Overall, the system evolved into a more disciplined and selective framework, with the final SDP model demonstrating strong capital protection and consistent alpha, though explicit loss-control mechanisms and stable predictions around change points are still needed for sustained profitability.

3.6 Summary

The final system could reliably identify strong trading signals but still produced losses (-14 percent on EBY, -32 percent on EBX). Analysis showed that the main issue was the lack of risk management: the agent held losing trades as long as winning ones, leading to occasional large losses that wiped

out its gains. The model learned how to enter good trades but not how to exit bad ones.

Although not yet profitable, the work successfully uncovered a usable alpha signal and pinpointed the exact failure point. Going forward, the focus will be on integrating explicit risk controls, especially dynamic stop-losses based on support and resistance, to prevent large drawdowns and move the strategy toward profitability.

4 Strategy Overview

The trading system developed in this work is a reinforcement learning–based execution engine built around a Proximal Policy Optimization (PPO) agent. Its goal is to learn profitable trading behaviour from a high-dimensional feature set derived from 15-second intraday data. Because financial markets are highly non-stationary, the system is embedded in a Walk-Forward Optimization framework, allowing the agent to retrain and adjust continually to recent market conditions rather than relying on patterns that may no longer be relevant.

4.1 Data Processing and Feature Engineering

The agent’s view of the market is constructed from a high-dimensional feature vector (over 100 columns) supplied at each timestep. This vector is designed to capture short-horizon market microstructure and includes a wide range of information such as:

- Raw OHLC price inputs,
- Price-Based (PB), Volume-Based (VB), and Band-Based (BB) technical indicators,
- Multiple windowed derivatives and differences of these indicators,
- Preprocessed columns such as rolling deltas from the PB/BB/VB families.

The purpose of this extensive feature set was to give the agent enough contextual information to detect subtle market patterns, including volatility clustering, emerging breakouts, and early signs of reversals.

4.2 PPO Agent Architecture

The training pipeline uses a rolling Walk-Forward Optimization (WFO) procedure to ensure the PPO agent learns from the most recent historical data without overfitting to any single period. No trading occurs during this stage; the WFO loop is used strictly for training and validation before deployment.

The process for each training batch is as follows:

Optimization Phase (Training Window)

- A portion of the historical window is allocated for training.

- The PPO agent learns exclusively from this segment, updating its actor–critic networks based on market patterns present in that recent slice of data.
- This phase allows the agent to internalize the short-term structure of the market before being evaluated.

Walk-Forward Phase (Validation Window)

- The remaining part of the window is used purely for validation.
- The agent is evaluated on this unseen segment without executing real trades.
- This phase checks whether the behaviours learned in the optimization segment generalize to the immediate future.
- Model performance on this walk-forward window determines how well the agent adapts to changing conditions.

This optimization → walk-forward cycle is repeated across all overlapping windows in the historical dataset, ensuring the agent trains progressively on the most recent information while being repeatedly tested just beyond its training region. Once all cycles are complete, the final trained model is saved and later used inside the trade loop for live inference.

4.3 Reward Engineering: The “Disciplined Trader” Reward

The agent’s learning is driven primarily by its reward function, which was designed to promote disciplined trading behaviour. The reward structure consists of three components. First, an entry penalty discourages unnecessary trades and forces the agent to be selective about when it enters the market. Second, a dense holding reward—based on bar-to-bar returns in the direction of the position—encourages the agent to stay with sustained trends rather than react to minor fluctuations. Finally, a closing bonus or penalty is applied when a trade exits, rewarding meaningful realized profits while imposing stronger penalties for losses to reinforce prudent risk management.

4.4 Resulting Behaviour of the Agent

The combination of the walk-forward setup, action stabilizer, and reward design produced a clear behavioural pattern during backtesting. The agent responded actively to market signals, showing it could interpret short-term patterns rather than being overwhelmed by noise. It was also capable of holding strong directional moves, confirming that the feature set carried meaningful predictive information. In several cases, average winning trades exceeded average losses, suggesting that some elements of the loss-aversion design were effective.

Despite these strengths, the strategy developed a hyperactive trading style that lacked effective risk control. The dense holding reward incentivized constant market exposure, leading to frequent low-quality trades and steady losses once costs and noise were included. The agent also showed equal patience toward both winners and losers; without strong

penalties for unrealized losses, losing trades were held too long, resulting in occasional large drawdowns that erased accumulated gains.

These outcomes indicate the need for a revised reward structure. Removing the dense holding reward in favor of a sparse, outcome-based approach would reduce over-trading. Adding penalties for unrealized losses and a small reward for remaining flat would encourage more selective, risk-aware behaviour and help prevent the large reversals that currently undermine performance.

5 Performance Metrics

The performance of the strategy was evaluated on two datasets: EBY and EBX. The results are summarized below.

5.1 EBY Data Performance

Total PnL: -14.58
 Max Drawdown: 3.40%
 Total Benchmark PnL: -3.43
 Win days percentage: 41.42%
 Loss days percentage: 48.51%

Table 1: Performance Statistics (EBY Data)

Statistic	Win		Loss	
	Portfolio	Benchmark	Portfolio	Benchmark
Count	111	125	130	143
Mean	0.405	0.466	-0.458	-0.431
Std Dev	0.403	0.428	0.510	0.428
Min	0.001	0.001	-3.395	-3.395
25% (Q1)	0.098	0.159	-0.554	-0.569
50% (Median)	0.273	0.338	-0.312	-0.312
75% (Q3)	0.630	0.635	-0.138	-0.132
Max	2.745	2.745	-0.000	-0.000

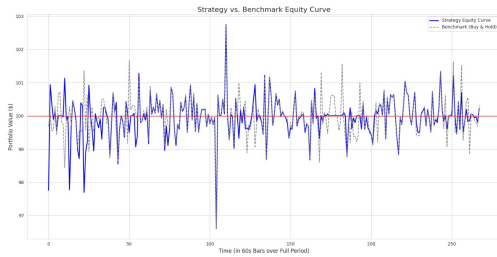


Figure 8: Performance Visualization for Portfolio vs Benchmark (EBY Data)

5.2 EBX Data Performance

Total PnL: -32.84
 Max Drawdown: 2.33%
 Total Benchmark PnL: -2.30
 Win days percentage: 35.12%
 Loss days percentage: 64.88%

Table 2: Performance Statistics (EBX Data)

Statistic	Win		Loss	
	Portfolio	Benchmark	Portfolio	Benchmark
Count	72	113	133	92
Mean	0.897	0.442	-0.732	-0.568
Std Dev	1.577	0.454	0.517	0.463
Min	0.006	0.000	-2.328	-2.288
25% (Q1)	0.231	0.167	-1.063	-0.821
50% (Median)	0.442	0.283	-0.630	-0.440
75% (Q3)	0.873	0.605	-0.303	-0.203
Max	12.372	2.982	-0.008	-0.006

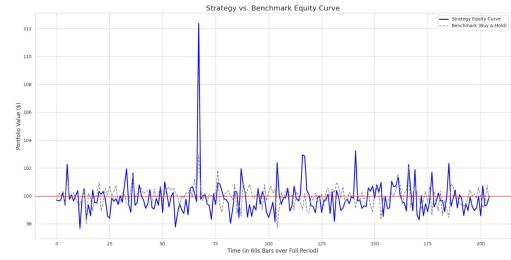


Figure 9: Performance Visualization for Portfolio vs Benchmark (EBX Data)

Analysis of Metrics: Based on the logs, this strategy’s unprofitability stems from a negative expectancy; its average loss per losing day (-0.458 points) is significantly larger than its average gain per winning day ($+0.405$ points). This is severely compounded by a failure to cut losses on long-held intraday trades, leading to catastrophic tail-risk events (like the -3.40% drawdown on Day 104) that wipe out any cumulative gains.

6 Future Scope

The experimentation phase produced a complete research and testing pipeline for high-frequency strategies. Our final PPO model, shaped by an improved reward design, adopted a patient, high-conviction trading style and successfully identified strong profit opportunities. However, backtests showed that these gains were offset by equally large losses, often caused by holding losing positions through market reversals, leading to an overall net loss. While the strategy is not yet profitable, the phase achieved its key objective by isolating a clear alpha signal and pinpointing the main failure mode. The next steps focus on adding explicit risk-management mechanisms to preserve gains and prevent such outlier losses.

6.1 Reward and Behavior Engineering

The PPO agent’s performance is highly dependent on the reward structure. While the current sparse, closing-only reward promotes patience, it also leads to symmetrical behaviour, causing the agent to hold both winning and losing trades for too long. Future work will focus on shaping behaviour to resemble that of a disciplined trader by introducing explicit stop-loss mechanisms, non-linear loss penalties,

and saturating profit rewards to encourage timely exits and better control of downside risk.

6.2 Risk Management and Stop Loss

Current risk management in the system is largely implicit, arising from parameter adjustments and reward shaping. The next step is to incorporate explicit, modular risk controls that operate alongside the trading logic. Position sizing will be adjusted dynamically based on volatility and signal confidence to reduce exposure in uncertain conditions and increase it when conviction is high. Additionally, volatility-aware trailing stops and support–resistance–based exit rules will be introduced to protect profits, limit reversals, and ensure more consistent retention of gains.

6.3 Better Feature Exploration

We intend to carry out a more thorough examination of the dataset to reveal extra structure and forecasting trends. This involves revisiting temporal dynamics, recognizing more insightful feature interactions, and exploring regime-specific patterns that might not have been completely addressed in the initial evaluation. A more thorough understanding of the dataset is expected, specifically EBX, to strengthen the predictive signal and guide the development of more robust, context-aware trading decisions.

References

- [1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
- [2] Haugen, K. K. (2016). Introduction. In *Risk Management* (pp. 13-25). Scandinavian University Press.